

EXPERIMENT NO. 9

Design a blog database and write scripts to seed it with mock data

Name: Rohit Pujari Roll No.: 28 Date: _____

Aim:

To design the SQLite database of “CodeJournal”, a developer blog with authors, posts, tags and threaded comments (replies to comments); to enforce data rules with constraints, speed up frequent queries with indexes and provide a statistics view; and to write a Node.js seed script (options --posts and --seed, a single transaction with rollback) and a query script that uses joins, aggregation and a recursive CTE, using Node.js's built-in node:sqlite module.

Theory:

Designing a database starts with the entities and their relationships. In CodeJournal an author writes many posts (one-to-many), a post has many tags and a tag belongs to many posts (many-to-many, stored in the junction table post_tags), and a post has many comments (one-to-many). A comment can also answer another comment, which is a one-to-many relationship of the comments table with itself (a self-referencing foreign key).

Tables

Table	Important columns	Constraints and purpose
authors	id, username, full_name, email, joined_on	username and email UNIQUE; CHECK on the e-mail pattern
posts	id, author_id FK, title, slug, body, status, views, published_at	slug UNIQUE (used in the URL); status IN ('draft', 'published'); views >= 0; a published post must have a date
tags	id, name	name UNIQUE COLLATE NOCASE, so "SQL" and "sql" are the same tag
post_tags	post_id FK, tag_id FK	Composite PRIMARY KEY (post_id, tag_id); junction table
comments	id, post_id FK, parent_id FK, name, body, created_at	parent_id NULL for a top-level comment, otherwise the id of the comment being answered

Integrity rules

- **PRAGMA foreign_keys = ON:** SQLite checks foreign keys only when this is switched on.
- **ON DELETE CASCADE:** deleting a post removes its tag links and comments; deleting a comment removes its replies.
- **Table-level CHECK:** CHECK (status = 'draft' OR published_at IS NOT NULL) compares two columns of the same row, which a column constraint cannot do.
- **Normal form:** author names and tag names are stored once and referenced by id, so the design is in third normal form. Dates are ISO text (YYYY-MM-DD), which sorts correctly as text.

Indexes and the view

SQLite automatically indexes primary keys and UNIQUE columns but not foreign keys, so indexes are added on posts(author_id), comments(post_id) and post_tags(tag_id), plus a composite index on posts(status, published_at) for “latest published posts”. EXPLAIN QUERY PLAN confirms that a search by author uses idx_posts_author instead of scanning the table (bottom of Figure 9.2). The view v_post_stats joins posts with authors and uses correlated sub-queries and GROUP_CONCAT() to return the comment count and tag list of every post, so programs can query it like a table.

Seeding strategy

- **Options:** --posts N and --seed S are read from process.argv; the file is deleted and re-created from schema.sql on every run.
- **Reproducible data:** a small seeded random generator (mulberry32) is used instead of Math.random(), so the same --seed always creates exactly the same rows.

- **Realistic rules:** about 80% of posts are published; drafts have no date and 0 views, which satisfies the CHECK constraint. Only published posts get comments, and about 40% of comments get a reply from the author.
- **Prepared statements** are created once and reused for every row, which is faster and safe from SQL injection. INSERT OR IGNORE skips a tag that was already chosen for the same post.
- **One transaction:** all inserts run between BEGIN and COMMIT; on any error ROLLBACK leaves the database empty instead of half-filled. Result: “Seeded codejournal.db (seed=7) in 4.2 ms” (Figure 9.1).

Queries

Query	SQL features	Result
1. Posts and views per author	LEFT JOIN with the condition in ON, GROUP BY, SUM()	Every author listed, busiest first
2. Most used tags	JOIN, GROUP BY, ORDER BY, LIMIT	Top five tags
3. Top posts	SELECT from the view v_post_stats	Title, views, comments, tags
4. Comment thread	WITH RECURSIVE, depth and sort path	Each reply printed under its parent

Important Code:

schema.sql - posts, threaded comments and the view

```
CREATE TABLE posts (
  id          INTEGER PRIMARY KEY,
  author_id   INTEGER NOT NULL REFERENCES authors(id) ON DELETE CASCADE,
  title       TEXT NOT NULL,
  slug        TEXT NOT NULL UNIQUE,
  body        TEXT NOT NULL,
  status       TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'published')),
  views       INTEGER NOT NULL DEFAULT 0 CHECK (views >= 0),
  published_at TEXT,
  CHECK (status = 'draft' OR published_at IS NOT NULL)      -- published posts need a date
);

CREATE TABLE comments (
  id          INTEGER PRIMARY KEY,
  post_id     INTEGER NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  parent_id   INTEGER REFERENCES comments(id) ON DELETE CASCADE, -- NULL = top-level, else a
  reply
  name        TEXT NOT NULL,
  body        TEXT NOT NULL,
  created_at  TEXT NOT NULL
);

CREATE VIEW v_post_stats AS
SELECT p.id, p.title, a.username AS author, p.status, p.views,
       (SELECT COUNT(*) FROM comments c WHERE c.post_id = p.id) AS comment_count,
       (SELECT GROUP_CONCAT(t.name, ', ') FROM post_tags pt JOIN tags t ON t.id = pt.tag_id
        WHERE pt.post_id = p.id) AS tag_list
FROM posts p JOIN authors a ON a.id = p.author_id;
```

seed.js - seeded random numbers and one transaction

```
let s = Number(args.seed || 7) >>> 0;
const rand = () => { s = (s + 0x6D2B79F5) >>> 0; let t = s; t = Math.imul(t ^ (t >>> 15), t | 1);
  t ^= t + Math.imul(t ^ (t >>> 7), t | 61); return ((t ^ (t >>> 14)) >>> 0) / 4294967296; };

db.exec('BEGIN'); // one transaction for all
rows
try {
  // ... insert authors, tags, posts, post_tags and comments with prepared statements ...
  db.exec('COMMIT');
} catch (err) {
  db.exec('ROLLBACK'); // nothing half-inserted
  console.error('Seeding failed, rolled back:', err.message);
  process.exit(1);
}
```

queries.js - recursive CTE for the comment thread

```
WITH RECURSIVE thread(id, depth, sort_path, name, body) AS (
  SELECT id, 0, printf('%05d', id), name, body FROM comments WHERE post_id = ? AND parent_id
  IS NULL
```

```

UNION ALL
SELECT c.id, t.depth + 1, t.sort_path || '/' || printf('%05d', c.id), c.name, c.body
FROM comments c JOIN thread t ON c.parent_id = t.id)
SELECT substr(' ', 1, depth * 3) || name AS commenter, body FROM thread ORDER BY
sort_path', busiest.post_id);

```

Output:

rohit@laptop: ~/wp-practicals

```

rohit@laptop:~/wp-practicals$ node exp09/seed.js --posts 40 --seed 7 && node exp09/queries.js
Seeded codejournal.db (seed=7) in 4.2 ms

```

(index)	Values
authors	5
tags	8
posts	40
post_tags	75
comments	97

1. Posts and total views per author (published only)

(index)	username	posts	total_views
0	'farah'	9	12890
1	'arjun.dev'	6	7689
2	'vikram'	7	6638
3	'rohitp'	5	5691
4	'sneha_k'	4	3304

Figure 9.1: Seeding with --posts 40 --seed 7 (row counts) and query 1

rohit@laptop: ~/wp-practicals

```

rohit@laptop:~/wp-practicals$ (output of queries.js continued)
2. Most used tags

```

(index)	tag	posts
0	'react'	15
1	'career'	14
2	'css'	11
3	'security'	10
4	'sql'	10

3. Top 5 posts from the view v_post_stats

(index)	title	author	views	comments	tags
0	'Why I Switched to Flexbox'	'farah'	2465	4	'career'
1	'Why I Switched to Database Indexes'	'vikram'	2408	3	'css, sql, react'
2	'5 Mistakes in React Hooks'	'arjun.dev'	2398	4	'security, career'
3	'Deep Dive into Database Indexes'	'farah'	2390	4	'react'
4	'A Beginner Guide to Git Rebase'	'farah'	2085	6	'security'

4. Comment thread of post 30 (recursive CTE)

(index)	commenter	body
0	'Aman'	'Very clear explanation, thanks!'
1	' author'	'Glad it helped!'
2	'Divya'	'Can you add an example with error handling?'
3	' author'	'Good question - I will update the post.'
4	'Divya'	'Very clear explanation, thanks!'
5	' author'	'Node 22, but it works on 20 too.'
6	'Pooja'	'Bookmarked for exams.'

```

Query plan for posts of one author:
SEARCH posts USING INDEX idx_posts_author (author_id=?)

```

Figure 9.2: Queries 2-4 (tags, the view, the recursive comment thread) and the query plan

Conclusion:

A normalised SQLite database for a developer blog was designed with foreign keys, CHECK constraints, a junction table for tags and a self-referencing comments table for replies. Indexes and a view supported the common queries. The seed script generated reproducible mock data inside a single transaction, and the query script demonstrated joins, aggregation, the view and a recursive CTE.